Bayesian Knowledge Tracing

13th Annual LearnLab Summer School, Pittsburgh, PA 2017

Michael Yudelson, Ph.D.,
Visiting Researcher, Carnegie Mellon University
Senior Research Scientist, ACT Inc.
PSLC Summer School 2006

Байесовское Отслеживание Знаний

13я Ежегодная Летняя Школа LearnLab, Питтсбург, ПА 2017

Михаил Юдельсон, Ph.D.,
Приглашенный Исследователь, Университет Карнеги Меллон
Старший Инженер-исследователь, ACT Inc.
Летняя Школа PSLC 2006

A little bit of context (1)

- Tutoring session
  - Human tutor or an Intelligent Tutoring System
- Student interacts with a problem
  - Single-step or multi-step
- Tutor tracks student's progress
- Tutor provides feedback
- Tutor gives advice

A little bit of context (2)

- Intelligent Tutoring System
  - Model tracing – interpret student behavior by comparing to the ***model of the problem***
  - Knowledge Tracing – infer student knowledge based on student performance; knowledge components (skills) associated with problem (steps)
- Cognitive Mastery
  - Deliver problems that address skills not mastered yet
  - Skill is considered mastered if inferred probabilistic value ≥ threshold

Agenda

- **Bayesian Knowledge Tracing (BKT)**
- BKT Variants
- Fitting BKT at Scale (`hmm-scalable`)
- Advanced Topics

Bayesian Knowledge Tracing

- **Goal**: for every skill the tutored problem (step) is associated with
  - Does the student know the skill? (mastery)
  - Will the student apply the skill correctly? (performance)
- Given the sequence of opportunities below (0-incorrect, 1-correct), answer the 2 questions above

1 0 1 0 1 1

Bayesian Knowledge Tracing
Definition

Bayesian Knowledge Tracing Model $\subseteq$ Hidden Markov Model $\subseteq$ Bayes Net $\subseteq$ Graphical Model

When modeling skill acquisition

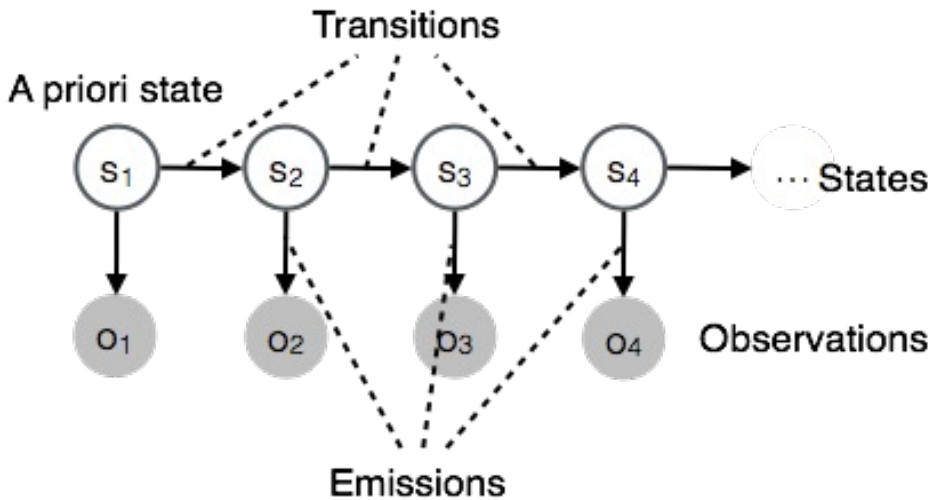
- One binary latent (unobserved) variable represents mastery (skill is ***known***|***unknown***), N=2
- One observed binary variable represents student performance (skill is applied ***correctly***|***incorrectly***), M=2

Data

- Time-series data of students applying skills to solve problems
  - Multiple time-series per skill from different students

Bayesian Knowledge Tracing
Representation

Unrolled view of BKT



Priors – Π

Known	Unknown
pL_0	1- pL_0

Transitions – A

	to Known	to Unknown
from Known	1-pF	pF
from Unknown	**pT**	1-pT

Emissions – B

	Correct	Incorrect
Known	1-pS	**pS**
Unknown	**pG**	1-pG

- 4 parameters
  - pL_o, pInit
  - pT, pLearn
  - pF, pForget
  - pS, pSlip
  - pG, pGuess
- Values ∈[0,1]
- Rows sum to 1
- Forgetting (pF) = 0

Bayesian Knowledge Tracing
Updating masteries, predicting

Parameters

Π	A		B	
pL_0	$\neg pF$	pF	$\neg pS$	pS
$\neg pL_0$	pT	$\neg pT$	pG	$\neg pG$

Outputs

L	C
pL	pC
$\neg pL$	$\neg pC$

Helpers

Le	Be(corr)	Be (incorr)
$pL\|e$	$\neg pS$	pS
$\neg pL\|e$	pG	$\neg pG$

	Matrix Version	**Simplified Computation**
1. Initialize	$L=\Pi$	$pL=pL_0$
2. Correct		
a. Predict	$C=L^T\times B$	$pC=pL\cdot(1-pS)+(1-pL)\cdot pG$
b. Pick Be	$Be=B[,1]$	
c. Compute Le	$Le=(L.\times Be)./(L^T\times Be)$	$p(L\|e=correct)=pL\cdot(1-pS)/[\,pL\cdot(1-pS)+(1-pL)\cdot pG\,]$
d. Update L	$L=Le\times A$	$pL=p(L\|e)\cdot(1-pF)+(1-p(L\|e))\cdot pT$
3. Incorrect (a and d same as in 2)		
b. Pick Be	$Be=B[,2]$	
c. Compute Le		$p(L\|e=incorrect)=pL\cdot pS/[\,pL\cdot pS+(1-pL)\cdot(1-pG)\,]$
4. Unknown (a-c same as in 2)		
d. Update L	$L=A^T\times L$	$pL=pL\cdot(1-pF)+(1-pL)\cdot pT$
5. Unknown/guess		
b. Pick Be	$Be=B[,argmax(C)]$	
continue as 2 or 3, depending on argmax(C)		

Legend

× – vector/matrix multiply
.× – multiply element by element
./ – divide element by element
Xe – X given evidence
L^T – transpose
X[,i] – i^{th} column of X

Bayesian Knowledge Tracing

http://yudelson.info/quickbkt.html

This is a quick helper interface for tracing/debugging/developing Bayesian Knowledge Tracing models [1]. It covers computing of the skill mastery (latent), performance (observation), as well as forward and backward variables [2].

Computations are done with the help of Numeric Javascript

pInit 0.5
pLearn 0.4
pForget 0.0
pSlip 0.1
pGuess 0.2

Parameters

Observations 1 0 1 0 1 1

Observations

0's and 1's separated by spaces

Compute | Update parameters. Expectation Maximization (Beta)

Likelihood of the data given the model: 0.006213451680

#	Right/ wrong	pKnown	pCorrect	Forward Variables		Backward Variables		Scale c_t	Fwd. Variables Scaled		Bwd. Variables Scaled	
				$\alpha_{t,i=1}$	$\alpha_{t,i=2}$	$\beta_{t,i=1}$	$\beta_{t,i=2}$		$\hat{\alpha}_{t,i=1}$	$\hat{\alpha}_{t,i=2}$	$\hat{\beta}_{t,i=1}$	$\hat{\beta}_{t,i=2}$
0†	n/a	0.500000	0.550000	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	
1	1	0.890909	0.823636	0.450000	0.100000	0.007290	0.029330	1.818182	0.818182	0.181818	0.015670	0.063044
2	0	0.703093	0.692165	0.049000	0.048000	0.072900	0.055028	5.670103	0.505155	0.494845	0.137935	0.104120
3	1	0.948525	0.863968	0.061380	0.005760	0.081000	0.215568	1.444742	0.914209	0.085791	0.134910	0.359040
4	0	0.818368	0.772858	0.006368	0.002765	0.810000	0.381600	7.351202	0.697280	0.302720	1.187555	0.559470
5	1	0.971798	0.880259	0.006727	0.000332	0.900000	0.480000	1.293899	0.952997	0.047003	1.161507	0.619470
6	1	0.996155	0.897309	0.006174	0.000040	1.000000	1.000000	1.136029	0.993592	0.006408	1.136029	1.136029

Bayesian Knowledge Tracing. Special Note on Predicting

- Updating L (pL) when the observation is known
  - In the tutor, the observation (correct/incorrect) is available for the pL update
    - Given 1..n skill attempts, predict n+1st
  - When comparing models making the observation available puts BKT at an unfair advantage
    - Especially, when comparing to logistic regression-based AFM
    - Less unfair when comparing to PFA (two learn rates)
  - Rule 4 from previous slide is too harsh
    - Matrix B (emissions) is ignored
    - Accuracy takes a considerable hit
  - Rule 5 is a compromise
    - Accuracy is better than under Rule 4 (but worse than under Rules 2&3)

Agenda

- Bayesian Knowledge Tracing (BKT)
- **BKT Variants**
- Fitting BKT at Scale (`hmm-scalable`)
- Advanced Topics

BKT Variants

- Standard BKT: one BKT model per skill
- Individualized BKT: parameterize students
  - Multiplex pInit (Pardos & Heffernan, 2010)
  - Fit BKT parameters within not across students (Lee & Brunskill 2012)
  - Fit per-skill and per-student parameters (Corbett & Anderson 1995; Yudelson et al. 2013)
- Accounting for item effects
  - Introduce an item node that affects performance (Pardos & Heffernan 2011)
- Dealing with noise in the data
  - From unigram observations to n-gram observations (Falakmasir et al. 2015)

Individualized BKT (1)

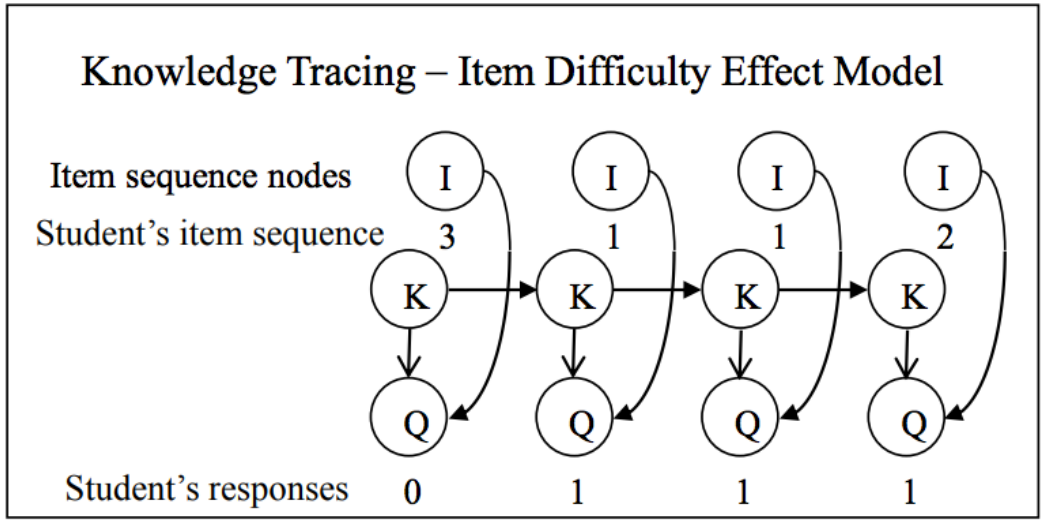
- Individualization – accounting for student-specific variation in performance
- Mulitplex pInit (Pardos & Heffernan, 2010)
  - Students are classified into low-pInit and high-pInit using first few attempts (often the first)
  - pKnown recomputed if classification changes from the default
- Fit BKT parameters within students (Lee & Brunskill 2012)
  - Hard to operationalize (chronic cold start problem)
  - Theoretical upper boundary
  - Resulting benefit of a more optimal problem selection is significant

Individualized BKT (2)

- Per-student and per-skill parameters
  - Standard 4 per-skill BKT parameters remain
  - Additional 4 per-student parameters
  - Pairing function combines the two
    - Corbett & Anderson 1995 – in odds space
    - Yudelson et al 2013 – in log-odds space
  - Fitting procedure.
    - Corbett & Anderson 1995 – fit once
    - Yudelson et al 2013 – block coordinate descent
  - Improvements are significant
  - Operationalization is tricky
    - Corbett & Anderson 1995 – log(student parameter)~raw error rate
    - Yudelson et al 2013 – fitting procedure pulls too much variance away from skill parameters
- Skill or Student, a limited test (Yudelson, 2016)
  - Parameter ~ $B_0 + B_1 \cdot \text{Skill} + B_2 \cdot \text{Student} + B_3 \cdot \text{Skill} * \text{Student}$
  - B_0 and B_3 are small (good)
  - Student weight B_2 is twice as high

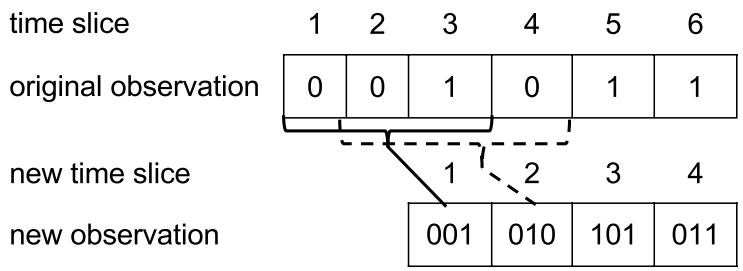
Accounting for item effects

- Introduce an item node that affects performance (Pardos & Heffernan 2011)
  - More complex HMM topology
  - Tested on ASSISTments and Cognitive Tutor data
  - Trained using BNT toolkit for Matlab/Octave



Dealing With Noise in the Data

- From unigram to n-gram observations: Feature compensation (often used in NL processing)



- >2 states and >2 observations

	to known	to unknown
from known	1	0
from unknown	0.4	0.6

	to 1	to 2	to 3	to 4
from 1	1	0	0	0
from 2	>0	>0	>0	0
from 3	>0	>0	>0	0
from 4	0	>0	>0	>0

- Best C-V'd accuracy achieved on KDD 2010 dataset

Model	Par/skill	C-V strat.	AIC	BIC	Acc.	RMSE
BKT	4	item	15380089	15478083	0.8609	0.3293
Spectral BKT	37	item	15853433	16758456	0.9208	0.2472
BKT	4	student	13947080	14045074	0.8659	0.3153
Spectral BKT	37	student	11553442	12458465	0.9196	0.2405

Agenda

- Bayesian Knowledge Tracing (BKT)
- BKT Variants
- **Fitting BKT at Scale (`hmm-scalable`)**
- Advanced Topics

`hmm-scalable`

- http://yudelson.info/hmm-scalable
- Toolkit for building Bayesian Knowledge Tracing (BKT) models using Big Datasets
- Extensively tested on educational data
- How big is a Big Dataset?
  - KDD Cup 2010 data. Challenge Set A/B – 9M/20M records 3K/6K students
  - 2010 Carnegie Learning Cognitive tutor data
    92.5 M records, 74 000 students (unfiltered)
  - Synthetic dataset – 902M records, 100K students
- How fast can `hmm-scalable` fit a model?
  - KDD Cup – 1 minute
  - 2010 Carnegie Learning unfiltered – 5 minutes (multi-core mode)
  - Synthetic data – 20 minutes (multi-core mode)

Obtaining and compiling `hmm-scalable`

Get the source code from IEDMS GitHub repository

```
$> git clone https://github.com/myudelson/hmm-scalable
```

OR download and unzip

Alternatively download an executable and readme from DataShop

External Tools > Fitting Bayesian Knowledge Tracing models at scale

`http://pslcdatashop.web.cmu.edu/ExternalTools?toolId=1`

Compile

```
$> make all
```

Operating system notes

- Linux – `gcc` with `Open MP` should be there by default
- Mac OS X – refer to hpc.sourceforge.net
- Windows – install `cygwin` with `awk`, **`make`**, `wc`, `time`, `unzip` commands

Getting KDD Cup 2010 Data

KDD Cup 2010 on hosted by LearnLab's DataShop [pslcdatashop.web.cmu.edu/KDDCup] Largest freely available dataset of learner data donated by Carnegie Learning, Inc.

- Download Bridge to Algebra challenge set bridge_to_algebra_2008_2009.zip (461Mb)
- ~20 000 000 rows, ~6 000 students

Data is placed in subfolder `d`, models will be in subfolder `m`, prediction files in subfolder `p`, code is in the root that contains these subfolders

Subset KDD Cup 2010 Data

Instead of unzipping a 5.3Gb training file, run:

```
$> time unzip -q -c d/bridge_to_algebra_2008_2009.zip
bridge_to_algebra_2008_2009_train.txt | awk "-F\t"
'BEGIN{OFS="\t"} {if(NR==1)next; skill=$20; gsub("~~",
"~", skill); skill=(skill==""")?".":($3"__"skill); print 2-
$14,$2,$3"__"$4,skill;}' > d/b89_train.txt
```

Reduces rich data to tab-delimited SSSS: success, student, step, skill(s). Multiple skills are ~-delimited. No skill label is denoted by . (dot).

Resulting text file is 2.9Gb.

Note: skills are prefixed with unit and section name. Identical skill labels could mean different things in different sections.

Training using `hmm-scalable`

Let's start from EM solver

```
$> ./trainhmm -s 1.1 -d ~ -m 1 -p 1 d/b89_train.txt
m/b89_modelEM.txt p/b89_predictEM.txt
```

Here, `-s 1.1` is an EM solver, `-d ~` is setting skill delimiter, `-m 1` asks for fit metrics, and `-p 1` requires predictions to be written into a file

It takes a little under 2.5 minutes, fitting alone 1 minute

```
trained model LL=6903034.9861273 (4490671.7596719),
AIC=13820821.972255, BIC=13930074.307203, RMSE=0.318739
(0.352797), Acc=0.870607 (s)
timing: overall 139.576931 sec, read 42.205482 sec, fit 60.997095
sec, predict 36.144902 sec
```

Here, LL – sum of negative log-likelihoods, AIC and BIT are Akaike and Bayesian Information Criterions, RMSE - root mean squared error, Acc – accuracy.

In braces are the RMSE and Accuracy for the rows without skill labels. Otherwise the metric is given for all rows, even if skill label was omitted.

Using Different "Solvers" (1)

- Baum-Welch (EM) `-s 1.1` – classical algorithm
- Stochastic Gradient Descent `-s 1.2`
  - Scales gradients so that the maximum step of 1 does not take any of them over 1 or below 0 (finds n to divide all parameters by 10^n)
  - Halves the step until either Wolfe Conditions are met (a step is far enough but not too far) or 20 attempts. If only first WC is met, us that (Nocedal & Wright, 1999).
  - Re-align parameters so that rows sum to 1 (project onto simplex)
  - Attempts to make large leaps in parameter space (Wolfe Conditions counter-balance that)
- Conjugate Gradient Descent `-s 1.3.[1-4]`
  - Make first step as in Stochastic Gradient Descent
  - Computes step direction on subsequent next step using formulas
    - Fletcher-Reeves `-s 1.3.1`
    - Polak-Ribière `-s 1.3.2`
    - Hestenes-Stiefel `-s 1.3.3`
    - Dai-Yuan `-s 1.3.4`
  - Makes large leaps in parameter space
- Gradient Descent using Lagrangian multipliers `-s 1.4`
  - Uses gradients in a manner that does not need realignment of parameters (Levinson et al., 1983)
  - Smooth[er] parameter changes, just like EM
- Temporarily excluded Barzilai-Borwein and exponentiated Gradient updates

Using Different "Solvers" (2)

Now let's try a Stochastic Gradient Descent solver

```
$> ./trainhmm -s 1.2 -d ~ -m 1 -p 1 d/b89_train.txt m/b89_modelGD.txt
p/b89_predictGD.txt
```

It takes a little longer

```
trained model LL=7230246.8188073 (4817883.5923708), AIC=14475245.637615,
BIC=14584497.972563, RMSE=0.328491 (0.368380), Acc=0.863745 (0.818622)
timing: overall 169.495414 sec, read 44.967394 sec, fit 100.809863 sec,
predict 23.490269 sec
```

The overall accuracy is a little worse 0.863745 (vs. 0.870607), just as the overall RMSE – 0.328491 (vs. 0.318739).

- How much is ***a little*** and what does it mean
  - Individualized BKT (Yudelson et al., AIED 2013)
    - Increment in the number of rows we are predicting correctly (~50K)
    - **When** the changes happen: earlier attempts – more valuable
  - Small Improvement for the Model Accuracy Big Difference for the Students (Yudelson & Ritter, AIED 2015)
    - Small difference in accuracy could mean tangible changes in how students work
    - Larger pLearn → faster mastery → fewer problems and less time

Comparing to the baseline

Accuracies/RMSE of the Majority Class prediction (here, always predicting "correct")

```
$> awk -F"\t" '{N++; C+=$1==1; SSE+= (1-$1)*(1-$1); }END{print C/N"\t"sqrt(SSE/N);}' d/b89_train.txt
```

Accuracy=0.861663 and **RMSE=0.371936**. Majority class accuracy is slightly worse than BKT models we've seen. The majority class RMSE is tangibly worse.

Note! BKT, traditionally, uses first attempt at a problem step. Further work until the student gets the step right is omitted.

- MC Accuracy ≈86% doesn't mean data is severely skewed or the Cognitive Tutor tutor is simple or all students are incredibly smart
  - 20 012 498 first attempts, 17 244 034 are correct = 86.1663% correct
  - 26 798 736 all attempts, 17 244 034 are correct = 64.3464% correct
- Contextual slip and guess BKT (Baker et al., 2010) uses all data.

Let forgetting happen

Default initial parameter values are: pL_0=0.5, 1-pF=1, pT=0.4, 1-pS=0.8, pG=0.2. To set defaults, one can specify a comma separated list via `-0 0.5,1.0,0.4,0.8,0.2`

Let's set a starting version of forgetting to 0.001

```
$> ./trainhmm -s 1.2 -d ~ -m 1 -p 1
-0 0.5,0.999,0.4,0.8,0.2 d/b89_train.txt
m/b89_modelGDpf0.01.txt
p/b89_predictGDpf0.01.txt
```

Π

Known	Unknown
pL_0	1- pL_0

A

	to Known	to Unknown
from Known	**1-pF**	pF
from Unknown	**pT**	1-pT

B

	Correct	Incorrect
Known	**1-pS**	pS
Unknown	**pG**	1-pG

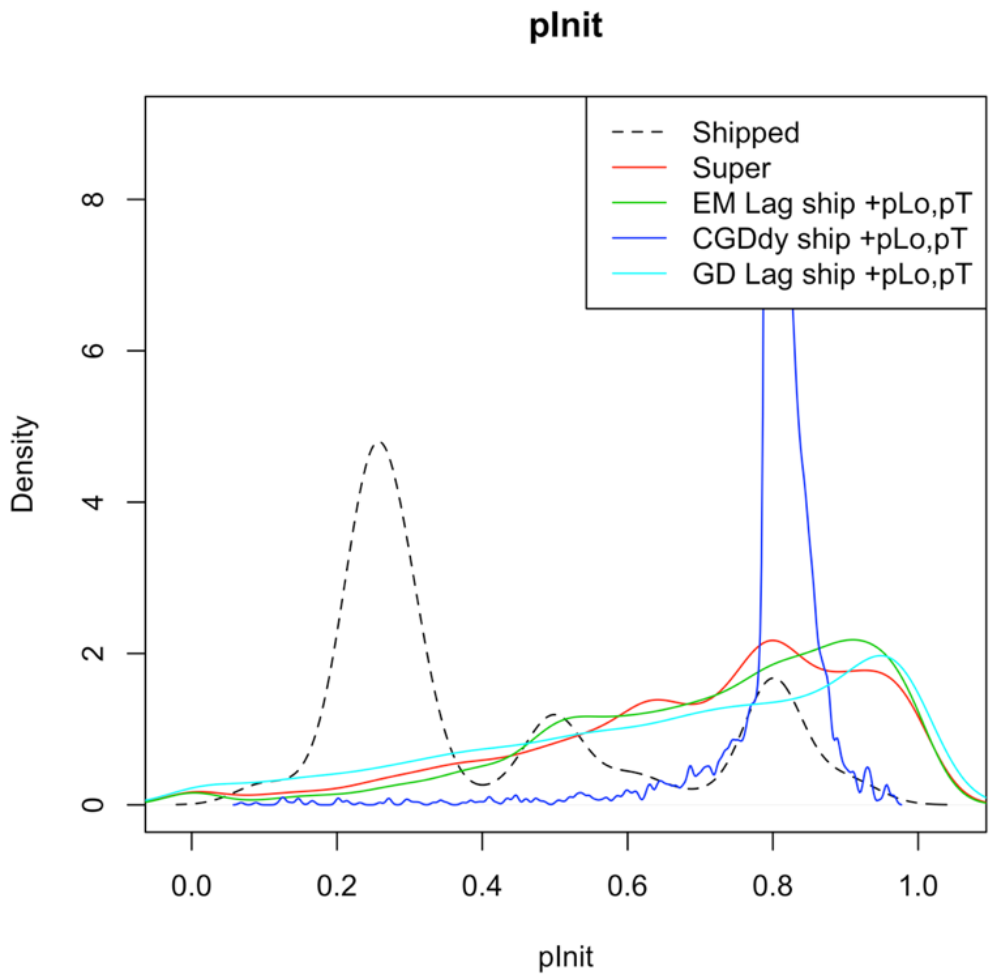
Small decrement in accuracy (rf. to 0.863745)

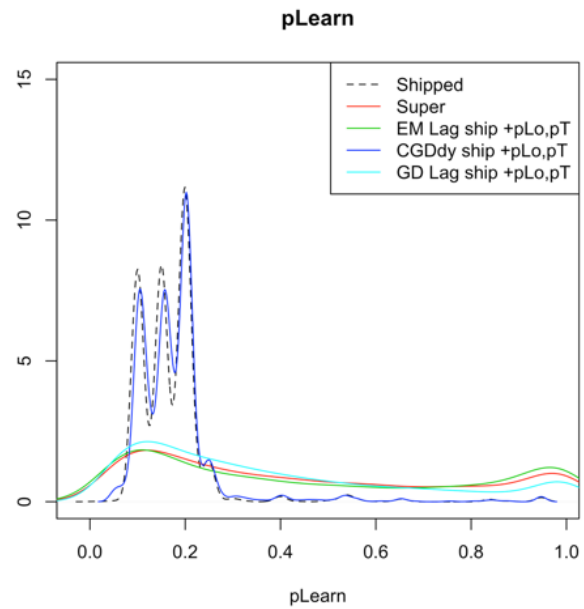
```
trained model LL=7747431.9193728 (5335068.6927902),
AIC=15509615.838746, BIC=15618868.173694, RMSE=0.340602
(0.387471), Acc=0.861669 (0.814925)
timing: overall 101.492081 sec, read 44.915405 sec, fit
30.866999 sec, predict 25.471139 sec
```

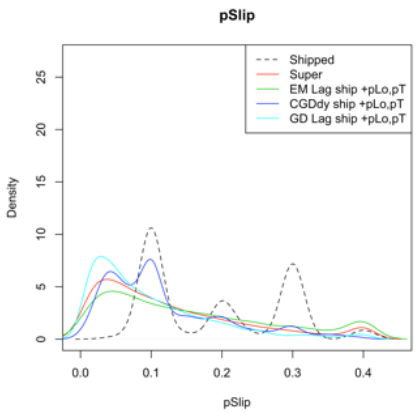
Why didn't we see drastic improvements?

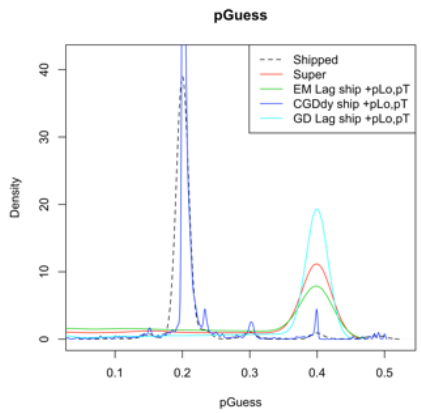
- We are using same set of starting parameters
  - Harder and easier skills *start* in the same place
  - Parameter space has multiple local optima
  - \+ Seeding pL_0 and pT using AFM – *pre-classify* (e.g., via modified `liblinear` in the IEDMS GitHub)
- We are not taking *context* into consideration
  - Learning and forgetting are likely non-uniform
  - Spacing effect: same day, next week, after Thanksgiving
  - \+ Multiplexing (switching) Transition (A) and Emission (B) matrices
- Presence of noise in the data (Falakmasir et al. 2015)

How different model parameters are?









Agenda

- Bayesian Knowledge Tracing (BKT)
- BKT Variants
- Fitting BKT at Scale (`hmm-scalable`)
- **Advanced Topics**

Advanced Topics
Prediction

- Updating probability distribution of states (PDS)
  - In tutor, previous observation (correctness) is always known
  - When assessing the model, revealing observation is not fair (e.g., AFM and PFA always *fly blind*)
- Controlling how update of the PDS is done

  `-U r,t` – for known observation **reveal** them for PDS update, for unknown observations – use **transition** matrix only (in Tutor)

  `-U g,g` – for known and unknown observations – **guess** most probable observation from PDO (Model Assessment)
- Probability distribution of observations (PDO) are always computed before the update and without *peeking*

Advanced Topics
Cross-Validation. Overview

- Cross-validation
  - Divide data into *n* pieces (folds)
  - Hide $1/n^{th}$ of the data, train on the rest
  - Predict the hidden data
  - Pick next fold, repeat
  - Combine *n* prediction folds, compute accuracy metric(s)
- Stratification
  - By student – $1/n^{th}$ of the students (whole student-skill sequences)
  - By-item – $1/n^{th}$ of the items (partial student-skill sequences)
  - Un-stratified – $1/n^{th}$ of the rows (partial student-skill sequences)
- Implementation
  - By-student stratification – simple
  - By-item/un-stratified – save $1/n^{th}$ of observations, and mark them as unknown

Advanced Topics
Cross-Validation. Example

- 5-fold student-stratified (default) cross-validation (rf. EM Acc.=0.870607)

```
./trainhmm -s 1.1 -d ~ -p 1 -b 1 -v 5 d/b89_train.bin m/b89_modelEMv.txt
p/b89_predictEMv.txt
...
5-fold cross-validation: LL=6932561.9857819, AIC=13879875.971564,
BIC=13989128.306512, RMSE=0.319513 (0.354041), Acc=0.870297 (0.830288)
timing: overall 206.000000 seconds, read 1.271167, fit 186.711080, predict
17.153612
```

- **10**-fold **item**-stratified cross-validation when predicting **incorrect observation**, and **save** the folds into a **file**

```
./trainhmm -s 1.1 -d ~ -p 1 -b 1 -P 1 -v 10,i,2,folds.txt,o
d/b89_train.bin m/b89_modelEMv2.txt p/b89_predictEMv2.txt
...
10-fold cross-validation: LL=19324495.2940466, AIC=38663742.588093,
BIC=38772994.923041, RMSE=0.321796 (0.357636), Acc=0.871027 (0.831588)
timing: overall 638.000000 seconds, read 1.312280, fit 618.701899,
predict 17.522869
```

- Saved folding can later be reused (read **in**) by using the following switch

```
-v 10,i,2,folds.txt,i
```

Advanced Topics
Parallel Execution

- Parallel code uses Open MP
  - Individual skills are fit in N (for me N=8) dynamically allocated parallel threads
- Checked-in code is in sequential state
  - Sequential code is easier to debug
  - Convert to parallel state by running `./seq2par.sh`
  - Convert back to sequential state `./par2seq.sh`
- To enable parallel execution use `-P 1` switch

```
 ./trainhmm -s 1.1 -d ~ -p 1 -b 1 -P 1 -m 1
d/b89_train.bin m/b89_modelEM.txt p/b89_predictEM.txt
...
timing: overall 33.484420 sec, read 1.232642 sec, fit
14.389842 sec, predict 36.632139 sec
```

Compare to sequential execution

```
timing: overall 139.576931 sec, read 42.205482 sec, fit
60.997095 sec, predict 36.144902 sec
```

Advanced Topics

What if there are just too many skills/students?

- Skill/student internal index variables
  - 4 byte signed integer
  - 2,147,483,647 (2 billion)
  - Default skill/student map is non-sparse
- 2010 Carnegie Learning unfiltered (Big Dataset)
  - 56000 students * 3700 skills → 56000*3700*2*4≅1.54Gb of memory for PSD alone
- `hmm-scalable` with BOOST* library's sparse 2D map
  - Compact representation
  - Currently not in public code

*http://www.boost.org

What I'd Like to add to `hmm-scalable`

- Multiple groupings of sequences
  - Student, Class, School – hierarchical effects
- Conjunctive BKT
  - Address multiple skills coding or other [item context] factors (FAST by González-Brenes et al, NIPS 2013)
- Continuous observations (e.g. time)
  - Math is already worked out
- Speed improvements
  - Optimized computation of forward-backward variables
  - NVIDIA Compute Unified Device Architecture (CUDA)
    - On this laptop: 384 GPU cores vs. 8 CPU cores

Thank You!

Acknowledgements

- Geoffrey J. Gordon & Kenneth R. Koedinger
  post-doctoral mentorship
- Steve Ritter
  keeping the project going as part of Big Data Optimization of Carnegie Learning Cognitive Tutor
- Mohamad Falakmasir & Young-Jin Lee
  attention to detail and asking hard questions